ФГБОУ ВО «Пензенский государственный университет»
Политехнический институт
Факультет вычислительной техники
Кафедра «Вычислительная техника»

ОТЧЁТ

По лабораторной работе №9
По дисциплине «Логика и основы
алгоритмизации в инженерных задачах»
На тему «Поиск расстояний в графе»

Выполнили: ст. гр. 25ВВВ1
Разживалов Н.А.
Цеплов Е.В.
Проверили: к.т.н., доцент Юрова О.В.
к.т.н., доцент Деев М.В.

Пенза 2026

Название

Поиск расстояний в графе

Цель работы

Научиться реализовывать функции поиска расстояний в ширину и в глубину. Сравнить время их выполнения.

Лабораторное задание

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа *G*. Выведите матрицу на экран.

2. Для сгенерированного графа осуществите процедуру поиска расстояний, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс queue из стандартной библиотеки C++.

3. Реализуйте процедуру поиска расстояний для графа, представленного списками смежности.

Задание 2

1. Реализуйте процедуру поиска расстояний на основе обхода в глубину.

2. Реализуйте процедуру поиска расстояний на основе обхода в глубину для графа, представленного списками смежности.

3. Оцените время работы реализаций алгоритмов поиска расстояний на основе обхода в глубину и обхода в ширину для графов разных порядков.

Ход работы

В теле main реализовали бесконечный цикл while, вовнутрь которого поместили конструкцию switch для выбора пользователем нужного действия.



```
int main() {
    setlocale(LC_ALL, "Russian");
    srand((unsigned)time(NULL));

    int** g = NULL;
    Node** gl = NULL;

    int n = 0;
    int choice = 0;
    int start;
    bool graphReady = false;

    while (1) {
        printf(
            "\nВыберите действие и введите его номер:\n"
            "1 - создать граф\n"
            "2 - задать случайные связи вершин графа\n"
            "3 - вывести матрицу смежности\n"
            "4 - вывести список смежности\n"
            "5 - найти расстояния BFS по матрице\n"
            "6 - найти расстояния BFS по списку\n"
            "7 - найти расстояния DFS по матрице\n"
            "8 - найти расстояния DFS по списку\n"
            "9 - выполнить все алгоритмы\n"
            "10 - измерить время работы алгоритмов\n"
            "11 - выход\n"
            "Ваш выбор: "
        );

        if (scanf_s("%d", &choice) != 1) {
            printf("Ошибка ввода\n");
            break;
        }

        switch (choice) {
        case 1: {
            int newN;

            printf("Введите количество вершин графа ");

            if (scanf_s("%d", &newN) != 1) {
                printf("Ошибка ввода\n");
                if (g != NULL) {
                    freeGraph(g, n);
                }
                return 1;
            }

            if (newN < 1 || newN > MAX_VERTICES) {
                printf("Введите число от 1 до %d\n", MAX_VERTICES);
                break;
            }

            int** newGraph = createGraph(newN);
```

Функции создания, заполнения случайными значениями, вывода и освобождения памяти графа для матрицы смежности и списков смежности взяли из прошлых работ.



```
void freeGraph(int** g, int n) {
    for (int i = 0; i < n; i++) {
        free(g[i]);
    }
    free(g);
}

int** createGraph(int n) {
    int** g = (int**)malloc(n * sizeof(int*));

    if (g == NULL) {
        return NULL;
    }

    for (int i = 0; i < n; i++) {
        g[i] = (int*)malloc(n * sizeof(int));

        if (g[i] == NULL) {
            for (int j = 0; j < i; j++) {
                free(g[j]);
            }

            free(g);
            return NULL;
        }
    }

    return g;
}
```

```
void printGraph(int** g, int n) {
    printf("    ");
    for (int j = 0; j < n; j++) {
        printf("%3d", j + 1);
    }
    printf("\n");

    for (int i = 0; i < n; i++) {
        printf("%3d ", i + 1);

        for (int j = 0; j < n; j++) {
            printf("%3d", g[i][j]);
        }

        printf("\n");
    }
}

void randGraph(int** g, int n) {
    if (g == NULL) {
        printf("Граф ещё не был создан\n");
        return;
    }

    for (int i = 0; i < n; i++) {
        g[i][i] = 0;
    }
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            g[i][j] = rand() % 2;
            g[j][i] = g[i][j];
        }
    }
    printGraph(g, n);
}
```

```
Node** createGraphList(int** g, int n) {
    Node** gl = (Node**)malloc(n * sizeof(Node*));

    for (int i = 0; i < n; i++) {
        gl[i] = NULL;
    }

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {

            if (g[i][j] == 1) {

                Node* newNode = (Node*)malloc(sizeof(Node));

                newNode->vertex = j;
                newNode->next = NULL;
                if (gl[i] == NULL) {
                    gl[i] = newNode;
                }
                else {
                    Node* current = gl[i];

                    while (current->next != NULL) {
                        current = current->next;
                    }

                    current->next = newNode;
                }
            }
        }
    }

    return gl;
}
```

```
void printGraphList(Node** gl, int n) {
    printf("\nСписок смежности:\n");

    for (int i = 0; i < n; i++) {
        printf("%d: ", i + 1);

        Node* current = gl[i];

        while (current != NULL) {
            printf("%d ", current->vertex + 1);
            current = current->next;
        }

        printf("\n");
    }
}

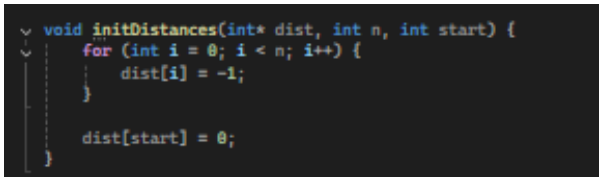
void freeGraphList(Node** gl, int n) {
    for (int i = 0; i < n; i++) {

        Node* current = gl[i];

        while (current != NULL) {
            Node* temp = current;
            current = current->next;
            free(temp);
        }
    }

    free(gl);
}
```

Написали функцию для хранения расстояний. Изначально обозначили все вершины необнаруженными



```
void initDistances(int* dist, int n, int start) {
    for (int i = 0; i < n; i++) {
        dist[i] = -1;
    }

    dist[start] = 0;
}
```

Реализовали функцию обхода в ширину. По заданию включили в неё класс очереди из стандартной библиотеки C++. Выбранной вершине устанавливается значение 0, а всем остальным -1. В очередь помещаются все вершины, затем проверяются соседи извлечённой вершины.

```
void bfsGraph(int** g, int n, int start, int* dist) {
    std::queue<int> q;

    initDistances(dist, n, start);
    q.push(start);

    while (!q.empty()) {
        int v = q.front();
        q.pop();

        for (int i = 0; i < n; i++) {
            if (g[v][i] == 1 && dist[i] == -1) {
                dist[i] = dist[v] + 1;
                q.push(i);
            }
        }
    }
}
```

Реализовали функцию обхода в ширину для списков смежности, оставив прежний принцип, но для другой логики.

```
void bfsList(Node** gl, int n, int start, int* dist) {
    std::queue<int> q;

    initDistances(dist, n, start);
    q.push(start);

    while (!q.empty()) {
        int v = q.front();
        q.pop();

        Node* current = gl[v];

        while (current != NULL) {
            int neighbor = current->vertex;

            if (dist[neighbor] == -1) {
                dist[neighbor] = dist[v] + 1;
                q.push(neighbor);
            }

            current = current->next;
        }
    }
}
```

Реализовали функцию обхода в глубину. Для этого сделали дополнительную функцию для подготовки к рекурсивным вызовам, которая инициализирует массив расстояний. Функция проверяет вершины, когда вершина, имеющая ребро с исходной, найдена, проверяются вершины, имеющие ребра уже с ней, затем, когда они заканчиваются, функция возвращается к прошлым вершинам и проверяет оставшиеся вершины аналогичным образом.

```
void dfsGraphRecursive(int v, int** g, int n, int* dist) {
    for (int i = 0; i < n; i++) {
        if (g[v][i] == 1 &&
            (dist[i] == -1 || dist[i] > dist[v] + 1)) {

            dist[i] = dist[v] + 1;
            dfsGraphRecursive(i, g, n, dist);
        }
    }
}

void dfsGraph(int** g, int n, int start, int* dist) {
    initDistances(dist, n, start);
    dfsGraphRecursive(start, g, n, dist);
}
```

Реализовали функцию обхода в глубину для списков смежности. Оставили исходный принцип, изменив логику для списков.

```
void dfsListRecursive(int v, Node** gl, int* dist) {
    Node* current = gl[v];

    while (current != NULL) {
        int neighbor = current->vertex;

        if (dist[neighbor] == -1 ||
            dist[neighbor] > dist[v] + 1) {

            dist[neighbor] = dist[v] + 1;
            dfsListRecursive(neighbor, gl, dist);
        }

        current = current->next;
    }
}

void dfsList(Node** gl, int n, int start, int* dist) {
    initDistances(dist, n, start);
    dfsListRecursive(start, gl, dist);
}
```

Реализовали функцию для подсчёта времени выполнения всех функций на 5 значениях. Создаётся матрица и список одного из 5 размеров. Затем подсчитывается время выполнения каждой функции.

```
void measureTime() {
    int sizes[] = { 10, 20, 30, 40, 50 };
    int sizeCount = sizeof(sizes) / sizeof(sizes[0]);

    printf("Сравнение времени работы алгоритмов\n");
    printf("Среднее время одного запуска, мс\n");

    for (int s = 0; s < sizeCount; s++) {
        int n = sizes[s];

        int** g = createGraph(n);

        if (g == NULL) {
            printf("Не удалось создать граф n=%d\n", n);
            continue;
        }

        for (int i = 0; i < n; i++) {
            g[i][i] = 0;
        }
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                g[i][j] = rand() % 2;
                g[j][i] = g[i][j];
            }
        }

        Node** gl = createGraphList(g, n);

        if (gl == NULL) {
            printf("Не удалось создать список смежности\n");
            freeGraph(g, n);
            continue;
        }

        int* dist = (int*)malloc(n * sizeof(int));

        if (dist == NULL) {
            printf("Ошибка выделения памяти\n");
            freeGraphList(gl, n);
            freeGraph(g, n);
            continue;
        }

        clock_t begin, end;
        double bfsM, bfsL, dfsM, dfsL;

        begin = clock();
        for (int k = 0; k < REPEATS; k++) {
            bfsGraph(g, n, 0, dist);
        }
        end = clock();
        bfsM = (double)(end - begin) * 1000.0 / CLOCKS_PER_SEC / REPEATS;

        begin = clock();
        for (int k = 0; k < REPEATS; k++) {
            bfsList(gl, n, 0, dist);
        }
        end = clock();
        bfsL = (double)(end - begin) * 1000.0 / CLOCKS_PER_SEC / REPEATS;

        begin = clock();
        for (int k = 0; k < REPEATS; k++) {
            dfsGraph(g, n, 0, dist);
        }
        end = clock();
        dfsM = (double)(end - begin) * 1000.0 / CLOCKS_PER_SEC / REPEATS;

        begin = clock();
        for (int k = 0; k < REPEATS; k++) {
            dfsList(gl, n, 0, dist);
        }
        end = clock();
        dfsL = (double)(end - begin) * 1000.0 / CLOCKS_PER_SEC / REPEATS;

        printf("%5d В ширину: матрица %f список %f. В глубину: матрица %f список %f\n", n, bfsM, bfsL, dfsM, dfsL);

        free(dist);
        freeGraphList(gl, n);
        freeGraph(g, n);
    }
```

По результатам видно, что быстрее выполняется поиск по спискам смежности и поиск алгоритмом поиска в ширину.

```
Ваш выбор: 10
Сравнение времени работы алгоритмов
Среднее время одного запуска, мс
   10 В ширину: матрица 0,008000 список 0,008000. В глубину: матрица 0,000000 список 0,000000
   20 В ширину: матрица 0,012000 список 0,012000. В глубину: матрица 0,010000 список 0,004000
   30 В ширину: матрица 0,020000 список 0,018000. В глубину: матрица 0,034000 список 0,018000
   40 В ширину: матрица 0,028000 список 0,024000. В глубину: матрица 0,094000 список 0,042000
   50 В ширину: матрица 0,036000 список 0,032000. В глубину: матрица 0,198000 список 0,088000
```

Псевдокод

```
ПРОЦЕДУРА INIT_DISTANCES(dist, n, start)
НАЧАЛО
   ДЛЯ i ОТ 0 ДО n - 1
      dist[i] := -1
   КОНЕЦ ЦИКЛА
   dist[start] := 0
КОНЕЦ ПРОЦЕДУРЫ

ПРОЦЕДУРА BFS_GRAPH(g, n, start, dist)
НАЧАЛО
   ВЫЗВАТЬ INIT_DISTANCES(dist, n, start)
   СОЗДАТЬ ПУСТУЮ ОЧЕРЕДЬ Q
   ДОБАВИТЬ start В КОНЕЦ Q
   ДО ТЕХ ПОР, ПОКА Q НЕ ПУСТА
      v := ПЕРВЫЙ ЭЛЕМЕНТ Q
      УДАЛИТЬ ПЕРВЫЙ ЭЛЕМЕНТ Q
      ДЛЯ i ОТ 0 ДО n - 1
         ЕСЛИ g[v][i] = 1 И dist[i] = -1 ТО
            dist[i] := dist[v] + 1
            ДОБАВИТЬ i В КОНЕЦ Q
         КОНЕЦ ЕСЛИ
      КОНЕЦ ЦИКЛА
   КОНЕЦ ЦИКЛА
КОНЕЦ ПРОЦЕДУРЫ

ПРОЦЕДУРА BFS_LIST(gl, n, start, dist)
НАЧАЛО
   ВЫЗВАТЬ INIT_DISTANCES(dist, n, start)
   СОЗДАТЬ ПУСТУЮ ОЧЕРЕДЬ Q
   ДОБАВИТЬ start В КОНЕЦ Q
   ДО ТЕХ ПОР, ПОКА Q НЕ ПУСТА
      v := ПЕРВЫЙ ЭЛЕМЕНТ Q
      УДАЛИТЬ ПЕРВЫЙ ЭЛЕМЕНТ Q
      current := gl[v]
      ДО ТЕХ ПОР, ПОКА current ≠ NIL
         neighbor := current.vertex
         ЕСЛИ dist[neighbor] = -1 ТО
            dist[neighbor] := dist[v] + 1
            ДОБАВИТЬ neighbor В КОНЕЦ Q
         КОНЕЦ ЕСЛИ
         current := current.next
      КОНЕЦ ЦИКЛА
   КОНЕЦ ЦИКЛА
КОНЕЦ ПРОЦЕДУРЫ
```

```
ПРОЦЕДУРА DFS_GRAPH(g, n, start, dist)
НАЧАЛО
   ВЫЗВАТЬ INIT_DISTANCES(dist, n, start)
   ВЫЗВАТЬ DFS_GRAPH_RECURSIVE(start, g, n, dist)
КОНЕЦ ПРОЦЕДУРЫ

ПРОЦЕДУРА DFS_GRAPH_RECURSIVE(v, g, n, dist)
НАЧАЛО
   ДЛЯ i ОТ 0 ДО n - 1
      ЕСЛИ g[v][i] = 1 И
        (dist[i] = -1 ИЛИ dist[i] > dist[v] + 1)
      ТО
         dist[i] := dist[v] + 1
         ВЫЗВАТЬ DFS_GRAPH_RECURSIVE(i, g, n, dist)
      КОНЕЦ ЕСЛИ
   КОНЕЦ ЦИКЛА
КОНЕЦ ПРОЦЕДУРЫ

ПРОЦЕДУРА DFS_LIST(gl, n, start, dist)
НАЧАЛО
   ВЫЗВАТЬ INIT_DISTANCES(dist, n, start)
   ВЫЗВАТЬ DFS_LIST_RECURSIVE(start, gl, dist)
КОНЕЦ ПРОЦЕДУРЫ

ПРОЦЕДУРА DFS_LIST_RECURSIVE(v, gl, dist)
НАЧАЛО
   current := gl[v]
   ДО ТЕХ ПОР, ПОКА current ≠ NIL
      neighbor := current.vertex
      ЕСЛИ dist[neighbor] = -1 ИЛИ
        dist[neighbor] > dist[v] + 1
      ТО
         dist[neighbor] := dist[v] + 1
         ВЫЗВАТЬ DFS_LIST_RECURSIVE(neighbor, gl, dist)
      КОНЕЦ ЕСЛИ
      current := current.next
   КОНЕЦ ЦИКЛА
КОНЕЦ ПРОЦЕДУРЫ
```

Результаты работы программы

```
     1  2  3  4  5  6  7
  1  0  0  1  0  1  0  1
  2  0  0  1  0  0  1  1
  3  1  1  0  0  0  0  1
  4  0  0  0  0  0  1  0
  5  1  0  0  0  0  0  1
  6  0  1  0  1  0  0  1
  7  1  1  1  0  1  1  0
Введите исходную вершину 1

Обход в ширину матрицы смежности
До вершины 1: 0
До вершины 2: 2
До вершины 3: 1
До вершины 4: 3
До вершины 5: 1
До вершины 6: 2
До вершины 7: 1

Обход в ширину списка смежности
До вершины 1: 0
До вершины 2: 2
До вершины 3: 1
До вершины 4: 3
До вершины 5: 1
До вершины 6: 2
До вершины 7: 1

Обход в глубину матрицы смежности
До вершины 1: 0
До вершины 2: 2
До вершины 3: 1
До вершины 4: 3
До вершины 5: 1
До вершины 6: 2
До вершины 7: 1

Обход в глубину списка смежности
До вершины 1: 0
До вершины 2: 2
До вершины 3: 1
До вершины 4: 3
До вершины 5: 1
До вершины 6: 2
До вершины 7: 1
```

```
Ваш выбор: 10
Сравнение времени работы алгоритмов
Среднее время одного запуска, мс
   10 В ширину: матрица 0,008000 список 0,008000. В глубину: матрица 0,000000 список 0,000000
   20 В ширину: матрица 0,012000 список 0,012000. В глубину: матрица 0,010000 список 0,004000
   30 В ширину: матрица 0,020000 список 0,018000. В глубину: матрица 0,034000 список 0,018000
   40 В ширину: матрица 0,028000 список 0,024000. В глубину: матрица 0,094000 список 0,042000
   50 В ширину: матрица 0,036000 список 0,032000. В глубину: матрица 0,198000 список 0,088000
```

Вывод

Научились реализовывать функции поиска расстояний в ширину и в глубину. Сравнили время их выполнения